

v1.1.0

OpenLcbCLib

Full OpenLCB / LCC Protocol Stack in C

Runs on any processor. No OS. No heap. Just clean C.

What is OpenLCB / LCC?

OpenLCB (Open Layout Control Bus) is an open, royalty-free standard for connecting model railroad accessories — throttles, signal towers, switch machines, lighting controllers, and more. The NMRA adopted it as LCC (Layout Command Control), making it the industry standard for interoperable layout electronics.

Nodes communicate over CAN bus or WiFi using a rich, well-specified protocol stack. Any manufacturer's node can talk to any other — out of the box, no configuration.

What is OpenLcbCLib?

The full OpenLCB/LCC protocol stack, written in plain C.

"No dynamic memory. No OS. No external dependencies. If your chip has a C compiler and a CAN peripheral, it can run OpenLcbCLib."

Key Features

Zero heap allocation — Every buffer is a static array sized at compile time. No malloc, no fragmentation, no surprises.

Full consist forwarding — Train Control listener management, P-bit forwarding, direction reversal, function linking, and source-skip loop prevention — all implemented.

OS-free by design — No RTOS required. A single `run()` call per main-loop tick drives the entire protocol engine.

Compliance-tested — 55 GoogleTest suites (2,147 tests) deliver 96.4% line, 98.0% function, and 91.2% branch coverage across the full source tree, and a guard audit compiles every feature-flag configuration. The OpenLCB OlcbChecker compliance suite runs fully automated against the macOS demo node.

Dependency injection — Hardware callbacks are function pointers wired at init time. Swap platforms by changing the driver layer — the protocol code is untouched.

Node Wizard — Browser-based project generator. Opens offline from the repo. Pick your platform, configure your node, download a ready-to-build ZIP.

Virtual multi-node — Host multiple logical nodes in one firmware image with full sibling loopback dispatch between them.

Stream Transport — Full stream initiation, buffer negotiation, flow-controlled data transfer, and stream completion — including config memory over streams.

MIT-friendly license — BSD 2-Clause. Use in open-source or commercial products with no royalties and no strings.

Protocol Coverage

Layer	Details
Frame Layer	Alias negotiation, conflict detection, CID/RID/AMD/AMR, CAN framing
Message Network	Verified Node ID, Protocol Support (PIP), OIR, Terminate Due to Error
SNIP	Simple Node Info Protocol — null-guard and null-terminator compliant
Event Transport	Producer / consumer, identify events, learn / teach
Datagram	Send, receive, Reply Pending with time estimate, timeout handling
Config Memory	Read, write, write-under-mask, lock, ACDI spaces, CDI / FDI delivery, DCC CV programming space (0xF8) with deferred replies
Broadcast Time	Query, set, immediate time, rollover, producer and consumer roles
Train Control	Speed, functions, e-stop, heartbeat, consist management, controller assign
Train Search	Search by address / flags, dynamic node creation, multi-protocol dispatch
Stream Transport	Initiation, buffer negotiation, flow-controlled data transfer, completion, config memory over streams
DCC Detector	Track occupancy detection, event-based reporting, identify producer support
Firmware Upgrade	Freeze / unfreeze, firmware write, bootloader handoff

Demo Platforms, Tools & Getting Started

Seven platform and transport combinations ship with ready-to-run example projects. Porting to a new CAN platform means writing just two driver files — a CAN hardware driver and an OpenLCB driver — then everything else runs unchanged.

Platform	Transport	Toolchain / IDE
ESP32	CAN (TWAI)	Arduino IDE, PlatformIO
ESP32	WiFi GridConnect	PlatformIO
Raspberry Pi Pico (RP2040)	MCP2517FD SPI	Arduino IDE (Philhower core)
STM32F4xx	CAN	STM32CubeIDE
TI MSPM0	MCAN	Code Composer Theia
dsPIC	CAN	MPLAB X
macOS	GridConnect over TCP	Xcode, PlatformIO

Architecture Highlights

- Static buffer pools sized at compile time — no malloc, no fragmentation
- Hardware access via nullable function-pointer interfaces (DI pattern)
- Single-tick main loop — no threads, no mutexes on single-core MCUs
- Alias validation: AMR scrub, FIFO invalidation, periodic re-verify

Unit Test Coverage

Metric	Coverage	Detail
Lines	96.4%	9,012 / 9,349
Functions	98.0%	813 / 830
Branches	91.2%	3,376 / 3,703

55 GoogleTest suites, 2,147 tests, cover the full source tree (utilities, CAN and TCP drivers, protocol engine), measured with gcovr on the typical configuration. The OlcbChecker compliance suite (frame, message network, SNIP, events, datagrams, config memory, CDI, FDI, broadcast time, train control and search) runs fully automated through test/olcbchecker_bridge.

Getting Started

1. **Quick Start (ESP32)** — Copy applications/arduino/esp32/BasicNode/ into your Arduino sketchbook and upload. Live on your layout in minutes.
2. **Node Wizard** — Open offline. Pick platform, configure node type and CDI, download a ready-to-build ZIP.
3. **New CAN platform** — Write a CAN hardware driver and an OpenLCB driver. The entire protocol engine runs unchanged.

Documentation & Repository

Quick Start Guide — Step-by-step ESP32 walkthrough

Developer Guide — Node Wizard, callbacks, CDI, project layout

Implementation Guide — State machines, protocol internals, architecture

API Reference — Full Doxygen-generated HTML

github.com/jimkueneman/OpenLcbCLib — BSD 2-Clause License

Built for model railroaders, by a model railroader.